

Reporte de resultados de prueba técnica

Andrea Cáceres M.

10 – Julio – 2025

Tabla de contenidos

Tabla de contenidos	2
Declaración de confidencialidad	3
Descargo de responsabilidad	3
Información de Contacto.....	3
Evaluación general.....	3
Resumen e Informe de Vulnerabilidades	4
Evidencia	6
H001: Claves o palabras secretas en el código.....	6
H002: Exposición de información innecesaria	6
H003: Validación de parámetros	6
H004: JWT Token sin expiración.....	6
H005: Parámetro user_id en formato alfanumérico	7
H006: Evitar envió de comandos de manera arbitraria	7
Escaneos e informes adicionales.....	8

Declaración de confidencialidad

Este documento es propiedad exclusiva de JELOU AI y de la autora. Este documento contiene información propietaria y confidencial. La reproducción, redistribución o utilización, total o parcial, en cualquier forma, requiere el consentimiento tanto de JELOU AI como de la autora.

JELOU AI puede compartir este documento con auditores bajo acuerdos de no divulgación para demostrar el cumplimiento de los requisitos de las pruebas realizadas.

Descargo de responsabilidad

Las conclusiones y recomendaciones reflejan la información recopilada durante la prueba técnica.

La autora priorizó la prueba técnica basándose en el top 10 del OWASP para identificar controles de seguridad más débiles que algún atacante podría explotar.

Información de Contacto

Nombre	Título	Información de contacto
Andrea Cáceres	Candidata	Correo: andreacaceresm@gmail.com

Evaluación general

Esta es una prueba técnica para evaluar las habilidades del candidato con el análisis de un código en Python y desplegado en un contenedor usando Docker. Todas las pruebas realizadas se basan en la Guía técnica de pruebas OWASP para sitios web.

Las fases de las actividades para esta prueba técnica realizadas se incluyen a continuación:

- Configuración – Se configuró un ambiente para desplegar Docker en una máquina iOS.
- Análisis – Se analizó el código de manera manual para identificar vulnerabilidades basados en el OWASP para sitios web.
- Remediación – Se creó un segundo archivo con las remediaciones de las vulnerabilidades encontradas.
- Reporte – Documento donde se reportan las vulnerabilidades encontradas.

Resumen e Informe de Vulnerabilidades

Hallazgo	Recomendación	Código(s) OWASP	Explicación
H001: Claves o palabras secretas en el código.	Crear variables de entorno o crear un archivo de variables y acceder a ellas mediante el código.	• A02:2021 • A08:2021	Almacenar claves en texto plano ayuda a la exposición de datos sensibles, debido a que si se llega a saber la <i>clave</i> o <i>palabra secreta</i> puede dar paso a ataques de fuerza bruta.
H002: Exposición de información innecesaria	Mostrar solo información necesaria y no información del servidor o claves.	• A05:2021 • A06:2021	Mostrar información adicional del servidor o de los recursos, puede causar que un atacante busque vulnerabilidades específicas del software.
H003: Validación de parámetros	Modificación de la consulta para evitar SQL Injection, ya sea usando parámetros o realizando una validación previa de los parámetros antes de realizar la consulta SQL.	• A03:2021	No realizar una adecuada validación de parámetros antes de cualquier otra acción, puede conllevar a inyección SQL y comprometer información sensible.
H004: JWT Token sin expiración	Agregar un tiempo (TTL) para evitar configuraciones de seguridad.	• A02: 2021 • A05:2021	No configurar un tiempo adecuado de sesión puede comprometer otros recursos que usen el

			token para realizar consultas. Además de que si un atacante llegará a robar ese token, puede usarlo con malas intenciones.
H005: Obtención de información sin validación previa	Tipos de consulta deberían realizarse enviando el token de sesión para más seguridad.	• A01:2021 • A06:2021	Permitir que se puedan realizar consultas a un API que contenga información sensible sin envío de algún tipo de token de inicio de sesión para autenticar que el usuario está permitido ver esa información.
H006: Evitar envío de comandos de manera arbitraria	Validar la entrada y evitar que se puedan enviar comandos peligrosos al servidor. Limitar los comandos ya sea por medio de una lista o de un arreglo.	• A03:2021 • A05:2021	Permitir que un usuario pueda enviar comandos al servidor es un fallo de seguridad grave debido a que permitirá que tenga un nivel control al servidor.

Evidencia

H001: Claves o palabras secretas en el código

```
SECRET_KEY = "supersecret"
DB_PATH = "./users.db"

# Database setup
@app.on_event("startup")
def init_db():
    conn = sqlite3.connect(DB_PATH)
    cursor = conn.cursor()
    cursor.execute("CREATE TABLE IF NOT EXISTS users (id INTEGER PRIMARY KEY AUTOINCREMENT, username TEXT, password TEXT)")
    cursor.execute("INSERT OR IGNORE INTO users (id, username, password) VALUES (1, 'admin', 'admin123')")
    conn.commit()
    conn.close()
```

H002: Exposición de información innecesaria

GET http://192.168.100.217:8000/health Send

Status: 200 OK Size: 158 Bytes Time: 24 ms

Response

```
{
  "status": "ok",
  "hostname": "376c33b0620e",
  "platform": "Linux-6.10.14-linuxkit-aarch64-with-glibc2.36",
  "python_version": "3.10.18",
  "secret_key_hint": "super*****"
}
```

H003: Validación de parámetros

POST http://192.168.100.217:8000/login Send

Status: 200 OK Size: 136 Bytes Time: 25 ms

Response

```
{
  "token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJ1c2V4X2lkIjoxCj1c2VybWVtZS16ImFkbWwluIn0.6T__1Pg7l2j-YnQ5L0xJEhDbGTCGHEk8agt5fndH7-U"
}
```

H004: JWT Token sin expiración

```
if user:
    token = jwt.encode({"user_id": user[0], "username": user[1]}, SECRET_KEY, algorithm="HS256")
    return {"token": token}
raise HTTPException(status_code=401, detail="Invalid credentials")
```

H005: Parámetro user_id en formato alfanumérico

The screenshot shows a REST client interface. The request is a GET to `http://192.168.100.217:8000/profile/1`. The response status is 200 OK, with a size of 27 Bytes and a time of 31 ms. The response body is a JSON object: `{ "id": 1, "username": "admin" }`. Red boxes highlight the `profile/1` in the URL and the response JSON.

```
GET http://192.168.100.217:8000/profile/1
```

```
{
  "id": 1,
  "username": "admin"
}
```

H006: Evitar envío de comandos de manera arbitraria

The screenshot shows a REST client interface. The request is a POST to `http://192.168.100.217:8000/exec`. The request body is a JSON object: `{ "cmd": "whoami; ls -la; echo hello; cd /var/www/html/; touch prueba.txt" }`. The response status is 200 OK, with a size of 30 Bytes and a time of 20 ms. The response body is a JSON object: `{ "message": "Command executed" }`. Red boxes highlight the `cmd` field in the request body and the `message` field in the response.

```
POST http://192.168.100.217:8000/exec
```

```
{
  "cmd": "whoami; ls -la; echo hello; cd /var/www/html/; touch prueba.txt"
}
```

```
{
  "message": "Command executed"
}
```

Escaneos e informes adicionales

La candidata realizó dos escaneos adicionales con los comandos *gobuster* y *dirsearch*. A continuación, se presenta los resultados de los comandos mencionados.

Usando el comando *gobuster* se pudo ver más directorios de los que GitHub mostraba como /docs y /redoc

```
danera@kali: ~/Documents/PruebasTecnicas/JelouAI x danera@kali: ~/Documents/PruebasTecnicas/JelouAI x
(danera@kali)-[~/Documents/PruebasTecnicas/JelouAI]
$ gobuster dir -u http://192.168.100.217:8000/ -w /usr/share/wordlists/dirbuster/directory-list-2.3-medium.txt

Gobuster v3.6
by OJ Reeves (@TheColonial) & Christian Mehlmauer (@firefart)

[+] Url: http://192.168.100.217:8000/
[+] Method: GET
[+] Threads: 10
[+] Wordlist: /usr/share/wordlists/dirbuster/directory-list-2.3-medium.txt
[+] Negative Status codes: 404
[+] User Agent: gobuster/3.6
[+] Timeout: 10s

Starting gobuster in directory enumeration mode

/login (Status: 405) [Size: 31]
/docs (Status: 200) [Size: 931]
/health (Status: 200) [Size: 158]
/exec (Status: 405) [Size: 31]
/redoc (Status: 200) [Size: 888]
Progress: 220560 / 220561 (100.00%)

Finished

(danera@kali)-[~/Documents/PruebasTecnicas/JelouAI]
$
```

Usando el comando *dirsearch* se encontraron las siguientes rutas

```
(danera@kali)-[~/Documents/PruebasTecnicas/JelouAI]
$ dirsearch -u http://192.168.100.217:8000/
/usr/lib/python3/dist-packages/dirsearch/dirsearch.py:23: DeprecationWarning: pkg_resources is deprecated as an API. See https://setuptools.pypa.io/en/latest/pkg_resources.html
  from pkg_resources import DistributionNotFound, VersionConflict

dirsearch v0.4.3

Extensions: php, aspx, jsp, html, js | HTTP method: GET | Threads: 25 | Wordlist size: 11460
Output File: /home/danera/Documents/PruebasTecnicas/JelouAI/reports/http_192.168.100.217_8000/25-07-09_22-01-27.txt
Target: http://192.168.100.217:8000/

[22:01:27] Starting:
[22:01:33] 307 - 0B - /docs/ → http://192.168.100.217:8000/docs
[22:01:33] 200 - 931B - /docs
[22:01:34] 405 - 31B - /exec
[22:01:34] 200 - 158B - /health
[22:01:36] 405 - 31B - /login
[22:01:36] 307 - 0B - /login/ → http://192.168.100.217:8000/login
[22:01:37] 200 - 2KB - /openapi.json
[22:01:39] 200 - 888B - /redoc

Task Completed

(danera@kali)-[~/Documents/PruebasTecnicas/JelouAI]
$
```